Parte I: Guía Práctica para la creación de aplicaciones de video *streaming* empleando la placa de desarrollo ESP32-CAM-MB/ESP32-XIAO-S3

@author: *vlarobbyk*
Fecha de Creación: *07/octubre/2024*
Fecha de Actualización: *08/noviembre/2025*
Versión: *1.1*

En este tutorial se ilustrarán los pasos que se requieren para desplegar un servidor de *streaming* para los dispositivos ESP32-CAM-MB y ESP32-XIAO-S3 que emplean la cámara OV2640.

El tutorial se enfoca en la creación de tanto del servidor de *streaming* como de aplicaciones que empleen código nativo (C++) o Python para realizar el procesamiento de imágenes en OpenCV que se capturen desde los dispositivos ESP32-CAM-MB y ESP32-XIAO-S3.

¿Por qué emplear este tipo de dispositivos?

Resulta a más de novedoso muy práctico desarrollar determinados tipos de proyectos con estos dispositivos en virtud de sus características. En la Tabla 1 podemos observar las principales características de los mismos:

Tabla 1: Principales características de los dispositivos ESP32-CAM-MB y ESP32-XIAO-S3

ESP32-CAM-MB	ESP32-XIAO-S3
➢ Procesador: ESP32-D0WD-V3 ➢ WIFI con soporte de protocolos IEEE 802.11 b/g/n/e/i ➢ Bluetooth 4.2 BR/EDR and BLE (Bluetooth Low Energy) ➢ Memoria: ➢ Puertos E/S (IO): 9 ➢ Dimensiones: 40.5mm x 27mm x 4.5mm ➢ Voltaje de operación: • Conector Micro USB: 5V@180mA • Temperatura trabajo: 20ºC-85ºC ➢ Soporta varios modos de suspensión: • Flash off: 180mA@5V • Flash on and brightness max: 310mA@5V • Deep-Sleep: as low as 6mA@5V • Modern-Sleep: as low as 20mA@5V • Light-Sleep: as low as 6.7mA@5V	➢ Procesador: ESP32-S3R8 Xtensa LX7 dual-core @ 240MHz (máxima velocidad) ➢ WIFI con soporte de protocolos IEEE 802.11 b/g/n/e/i ➢ BLE: Bluetooth 5.0, Bluetooth mesh ➢ Memoria: 8Mb PSRAM & 8Mb Flash ➢ Puertos: 11 puertos GPIO ➢ Dimensiones: 21 x 17.8 mm ➢ Voltaje de operación: • Conector Tipo-C: 5V@19mA • Batería: 3.8V@22mA • Con WIFI activo: 100mA • Con BLE activo: 85mA • Temperatura trabajo: 40ºC-60ºC ➢ Soporta varios modos de suspensión: • Modem-sleep Model: 3.8V/25 mA • Light-sleep Model: 3.8V/2 mA • Deep Sleep Model: 3.8V/14 µA

Algunas de las principales aplicaciones IOT (*Internet of Things*) en las que se usan este tipo de dispositivos son las siguientes:

- Carga de imágenes en dispositivos para hogares inteligentes
- Monitoreo a través de redes inalámbricas
- Agricultura inteligentes
- Identificación inalámbrica de códigos QR
- Reconocimiento facial
- Robótica

Se recomienda revisar el respositorio de Github donde existe documentación adicional relacionada con los drivers para cámara ESP32:

https://github.com/espressif/esp32-camera

1 Instalación de herramientas

Como primer paso debemos instalar las herramientas necesarias para desarrollar la aplicación en Arduino, ya que el microcontrolador ESP32 es compatible con este entorno. En esta sección presentarán paso a paso los entornos y librerías que deben instalarse previamente.

1.1 Arduino IDE

El entorno de desarrollo que se utilizará es *Arduino IDE | 2.3.6*. Para instalarlo en sistema operativo basta con descargar la versión correspondiente desde el sitio oficial de Arduino® [enlace de descarga]. Una vez descargada la versión correspondiente al sistema operativo que estemos empleando, descomprimimos el fichero en la carpeta de nuestra preferencia, y ejecutamos el comando ***arduino-ide*** (en este tutorial se trabajará para sistemas GNU/Linux, siendo muy similar al caso de Microsoft Windows® para iniciar el entorno.

En el caso de usar distribuciones basadas en GNU/Ubuntu 24.04, se puede presentar el siguiente error de seguridad:

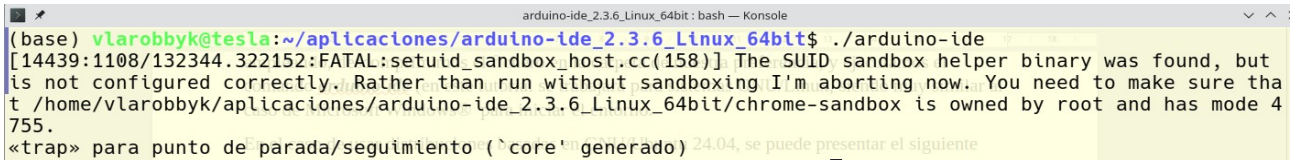


Figura 1: Error de ejecución del IDE Arduino 2.3.6 en sistemas operativos basados en GNU/Ubuntu 24.04

Esto se debe a que el IDE de Arduino debe ejecutarse como usuario sin privilegios de administrador (root) y esto entra en conflicto con el nuevo sistema de seguridad implementado en estos sistemas operativos (AppArmor).

Para solucionar este problema existen diversas opciones, sin embargo, podemos simplemente crear un perfil AppArmor para el IDE de Arduino. Para ello debemos crear el archivo ***usr.local.bin.arduino*** en la ruta ***/etc/apparmor.d/*** y colocar el siguiente código dentro:

Tabla 2: Código para la creación de un perfil AppArmor para la correcta ejecución del IDE de Arduino. Fuente: ***mariovaldez*** *[https://github.com/arduino/arduino-ide/issues/2429#issuecomment-2099775010]*

```
abi <abi/4.0>,
include <tunables/global>
profile arduino /home/vlarobbyk/aplicaciones/arduino-ide_2.3.6_Linux_64bit//arduino-ide
flags=(unconfined) {
    userns,
    include if exists <local/arduino>
}
```

Es importante observar que el código resaltado con amarillo corresponde a la ruta donde se haya instalado el IDE de Arduino, ya sea vía descarga del comprimido en formato ZIP (la más cómoda) o vía instalación de paquete. Debe verificar también el nombre del archivo ejecutable, en el caso de este tutorial es **arduino-ide**.

Una vez creado el archivo, recargamos todos los perfiles del sistema de protección AppArmor con el siguiente comando y volvemos a cargar sin problema el IDE de Arduino:

```
sudo service apparmor reload
```

1.2 Acceso a puertos

El IDE de Arduino no podrá acceder al puerto USB donde está conectado el dispositivo si antes no damos acceso a dicho puerto al usuario del sistema operativo que ha lanzado el IDE.

Para brindar este acceso, debemos añadir al usuario actual al grupo **dialout** a través del siguiente comando (que debe ser ejecutado como **root**):

```
sudo usermod -a -G dialout vlarobbyk
```

Donde **vlarobbyk** corresponde al usuario actual. Usted deberá reemplazarlo por su nombre de usuario y **reiniciar el computador** para que los cambios tengan efecto.

1.3 Preparación del dispositivo

Como primer paso debemos colocar la antena WIFI que viene junto con el dispositivo ESP32. Para ello, simplemente conectamos la misma como se indica en la Figura 2 (haciendo una leve presión sin dañar la base de la misma ni el soporte):

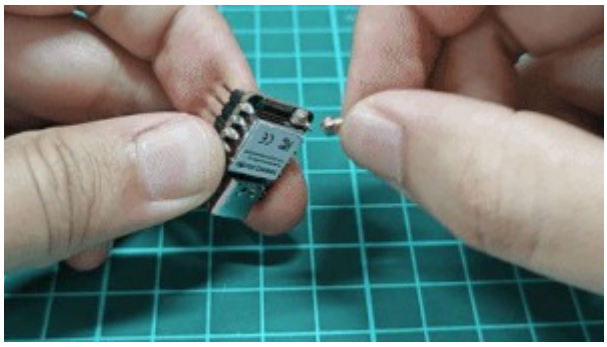


Figura 2: Ejemplo de colocación de la antena WIFI al módulo ESP32. Imagen tomada de: https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/

Una vez conectada la antena, procedemos a conectar el dispositivo al computador empleando para ello el cabe correspondiente (USB Micro o USB C, dependiendo de la versión del dispositivo).

1.4 Instalación de librerías en el Arduino IDE para el desarrollo de la aplicación servidor

A fin de poder programar la tarjeta ESP32, debemos instalar las librerías de soporte correspondiente. Para realizar este paso debemos ir al menú **File→ Preferences** y colocar el siguiente texto en la opción **Additional Boards Manager URLs:**

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

Luego de ello, damos clic en el botón “Ok”, como se indica en la Figura 3:

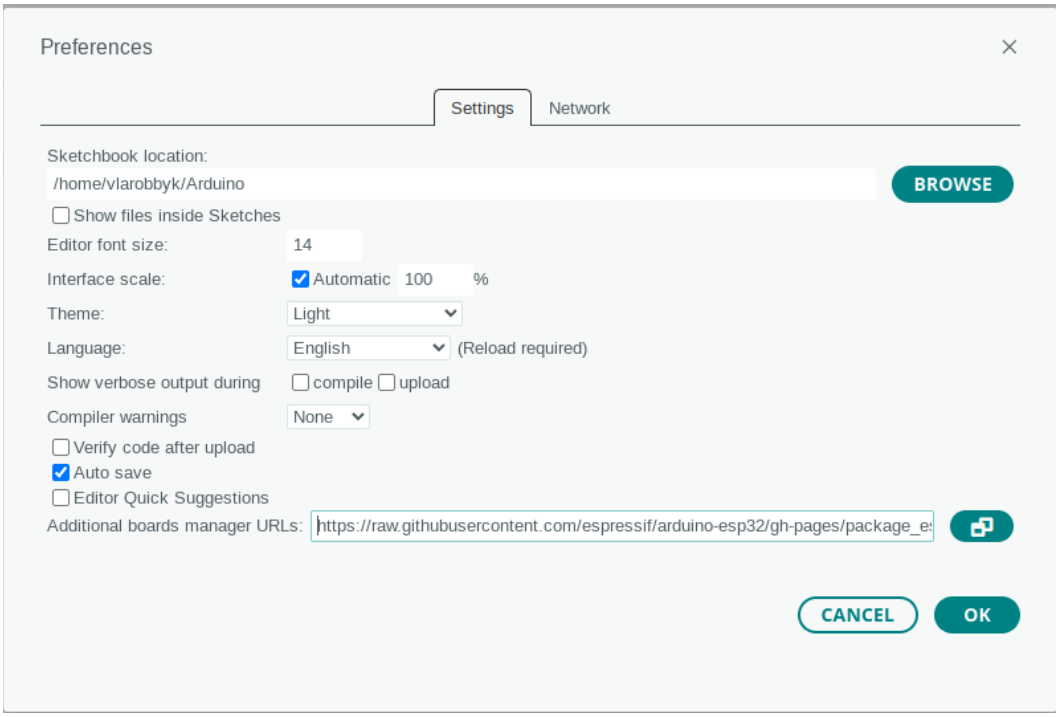
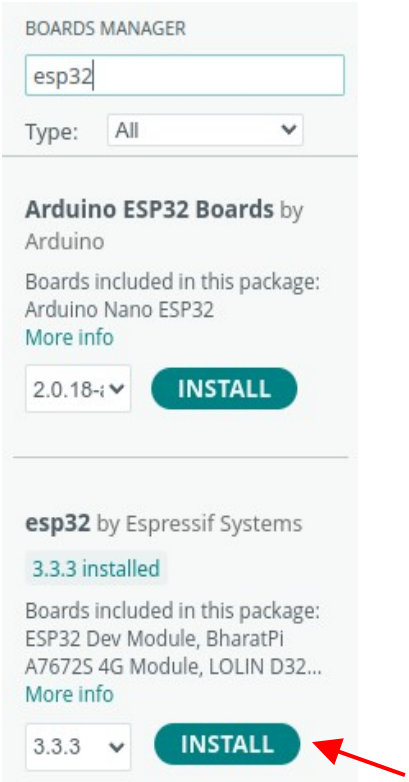


Figura 3: Adición de soporte para programar tarjetas ESP32 en el IDE de Arduino

Luego de ejecutar este paso, debemos al menú **Tools → Board → Boards Manager** y escribir **"ESP32"** en la caja de búsqueda, seleccionar la última versión y dar clic en el botón **"INSTALL"**, como se indica en la Figura 4:



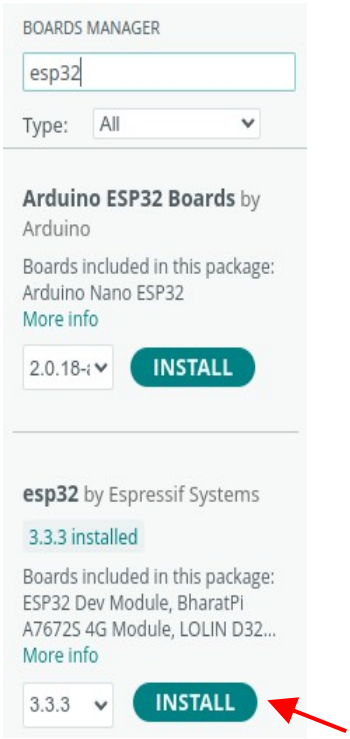


Figura 4: Instalación de las librerías para programar tarjetas ESP32

Una vez que se ejecute este paso, se instalarán las librerías que se requieren.

1.5 Selección de la tarjeta y el puerto de trabajo

Una vez conectado el dispositivo ESP32, deberemos seleccionar el modelo correspondiente empleando el menú **"Select Board"**. Si tenemos conectado un solo dispositivo será el único que aparezca al desplegar la lista, caso contrario debemos seleccionar el AI Thinker ESP32-CAM o el ESP32-XIAO-S3, como se indica en la Figura 5:

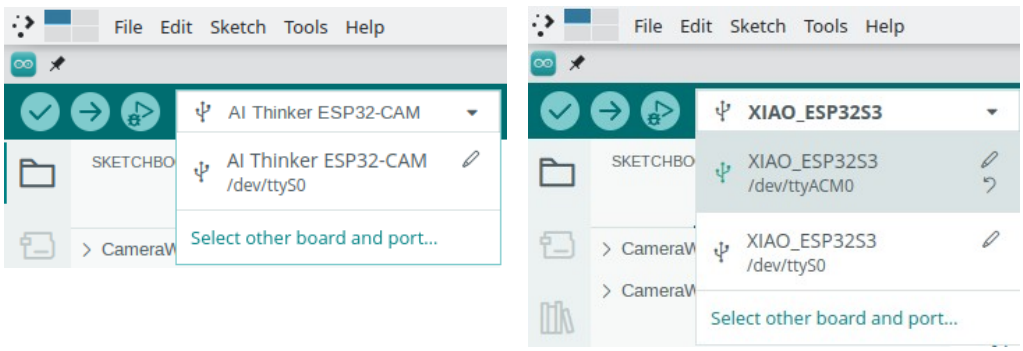


Figura 5: Selección del dispositivo ESP32 en el menú del IDE de Arduino (a la izquierda AI Thinker ESP32-CAM y a la derecha XIAO-S3)

Como se aprecia, nos muestra el nombre del dispositivo y el puerto, sin embargo, este no corresponde con el dispositivo que tenemos. Por ello, deberemos dar clic en la opción **"Select other board and port…"** , escribir en la caja de búsqueda **"esp32"** y ahí seleccionar el dispositivo correcto, en este caso **"ESP32-S3 Dev Board"**, como se indica en la Figura 6.

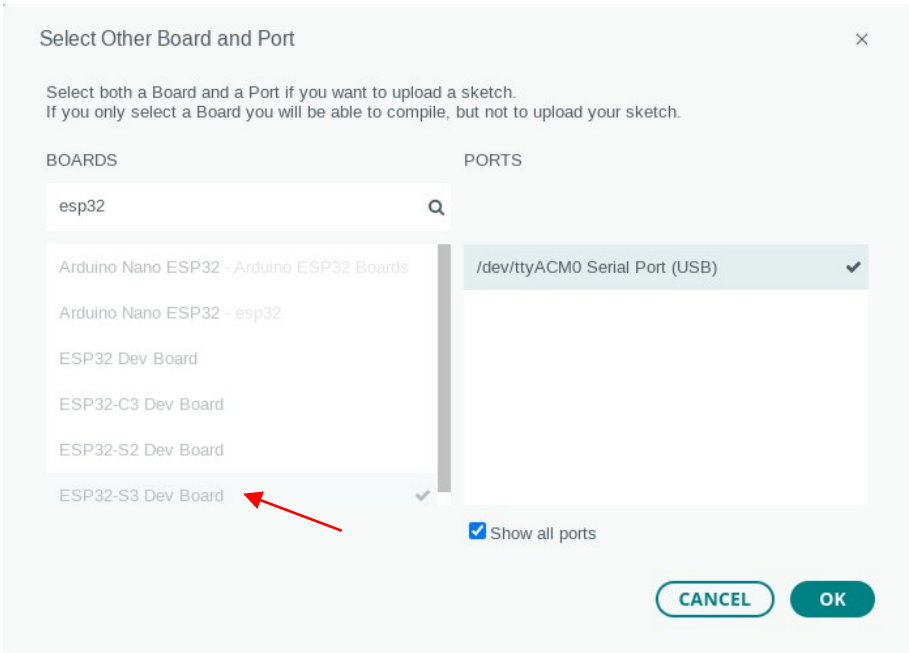


Figura 6: Selección del dispositivo ESP32-XIAO-S3 con el que se desarrolló el tutorial

Al dar clic en el botón **"OK"** se presentará un aviso que indica que debe instalarse el código **core** en la tarjeta seleccionada, deberemos dar clic en el botón **"YES"** (que normalmente aparece en la parte inferior derecha del IDE), como se indica en la Figura 7:

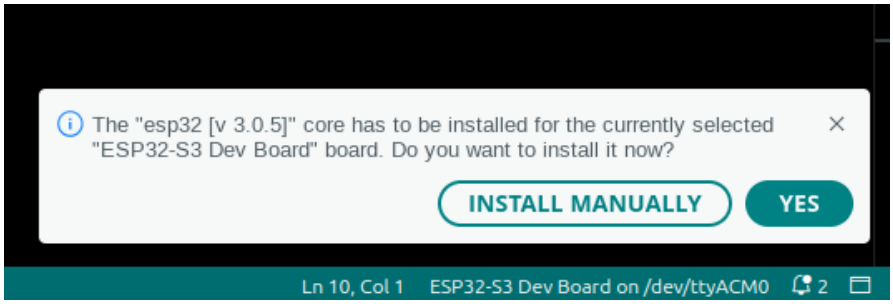


Figura 7: Solicitud de instalación del código core en la tarjeta seleccionada.

Al ejecutar esta paso, se procederá a instalar el código **core** en la tarjeta ESP32. Este proceso suele tardar varios minutos y si todo ha sido exitoso, deberá observar un mensaje que solicita **reseleccionar** la tarjeta, deberemos dar clic en el botón **"YES"** como indica la Figura 8:

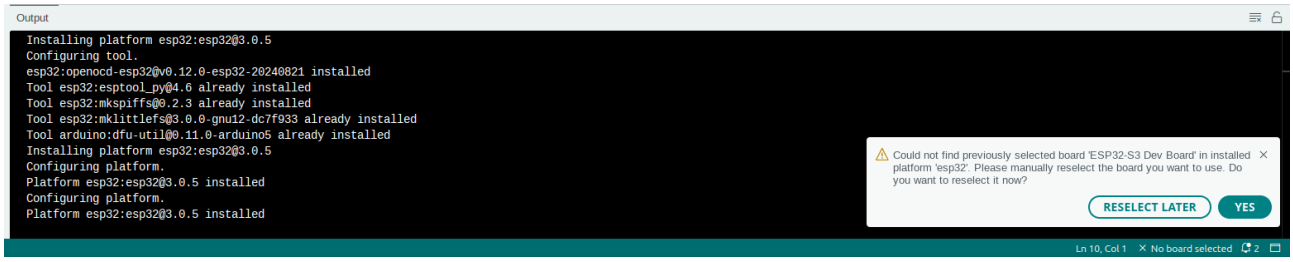


Figura 8: Re-selección de la tarjeta una vez instalada la librería core.

Al dar clic en el botón **“YES”** se despliega nuevamente el menú para seleccionar la tarjeta ESP32, y en esta ocasión veremos que ya está habilitada la tarjeta correcta. Seleccionamos el modelo correspondiente (en este caso **XIAO_ESP32S3**) y damos clic en el botón **“OK”** como se indica en la Figura 9:

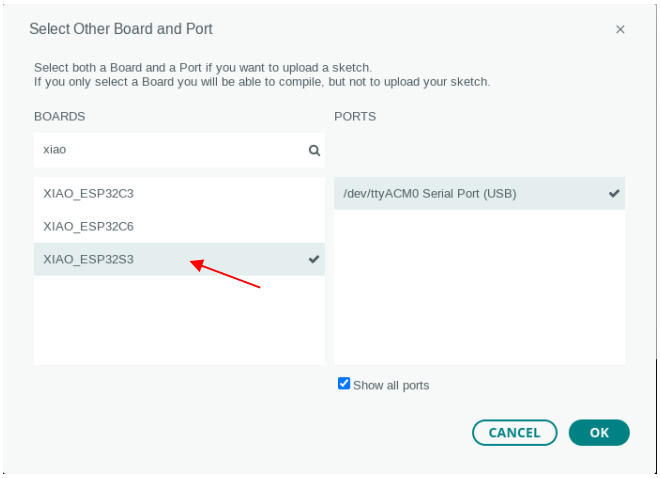


Figura 9: Re-selección de la tarjeta ESP32 para desarrollar el programa.

1.6 Instalación de la librería “Serial”

A fin de poder cargar los programas en la tarjeta ESP32, debemos instalar la librería “Serial” como primer paso. Para ello, primero cerramos el IDE de Arduino y luego deberemos abrir la terminal, activar el entorno virtual de **Conda** con el que se esté trabajando e instalar la librería **pyserial**, a través del siguiente comando:

```
pip install pyserial
```

Con el entorno virtual de **Python** abierto, lanzamos nuevamente el IDE de Arduino, como se observa en la Figura 10:



Figura 10: Carga del IDE de Arduino con el entorno virtual de Python activo.

En este caso, el entorno virtual de Python se llama **opencv**, Usted deberá trabajar con el entorno de su preferencia.

1.7 Carga del servidor de video streaming

Dentro del IDE de Arduino ya existen varios ejemplos para el manejo de diversos dispositivos. Por ello, emplearemos como base el ejemplo para manejo de cámaras en ESP32. Deberemos abrir la siguiente ruta:

File → Examples → ESP32 → Camera → CameraWebServer

Una vez abrimos el ejemplo, se cargará automáticamente todo el código que se requiere para lanzar el servidor de *streaming* de video.

Debemos modificar la línea que corresponde con el modelo que tenemos instalado, en este caso comentamos la línea de la cámara que viene por defecto y quitamos el comentario de la siguiente línea:

```
#define CAMERA_MODEL_XIAO_ESP32S3 // Has PSRAM
```

En el caso de usar el modelo ESP32-XIAO-S3. Si por el contrario, usamos el modelo ESP32-CAM-MB, deberemos quitar el comentario a la siguiente línea:

```
#define CAMERA_MODEL_AI_THINKER // Has PSRAM
```

Una vez realizado este cambio, debemos colocar las credenciales de la red WIFI con la que estamos trabajando, a fin de que la tarjeta pueda desplegar el servidor de *video streaming*. Modificaremos las siguientes líneas, de acuerdo a los datos de nuestra red (nombre de la red y contraseña, respectivamente):

```
const char *ssid = "**********";
const char *password = "**********";
```

Guardamos el archivo en la carpeta donde se vaya a almacenar el proyecto y damos clic en el botón **“Verify”**  .

Ello permitirá verificar que se compile el archivo *sketch* de Arduino. Posterior a esto, deberemos dar clic en el botón **“Upload”**  para que el código sea cargado en la tarjeta ESP32.

Para verificar que todo hay sido exitoso, deberá observar el mensaje **“Writing at 0X… (100%)”** en ventana **Output**. En el monitor serial (**Tools → Serial Monitor**) podrá observar la IP en la que está sirviendo el dispositivo ESP32, como se indica en la Figura 11:

“Camera Ready! Use http://DIRECCIÓN_IP to connect

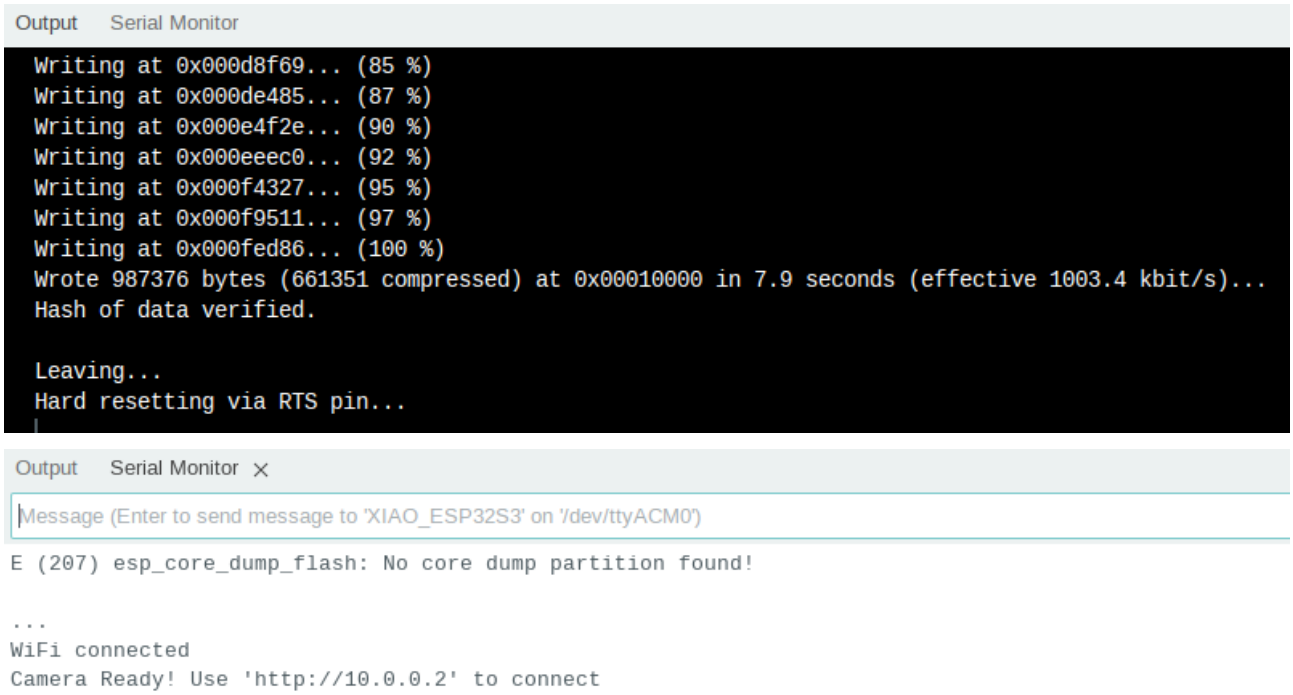


Figura 11: Resultado de la correcta compilación del código para desplegar el servidor de video streaming en el ESP32.

Si abrimos la dirección IP en el navegador, podremos observar todos los parámetros de configuración de la cámara, y al dar clic en el botón **“Start Stream”** se desplegará el vídeo en la página, como se indica en la Figura 12:

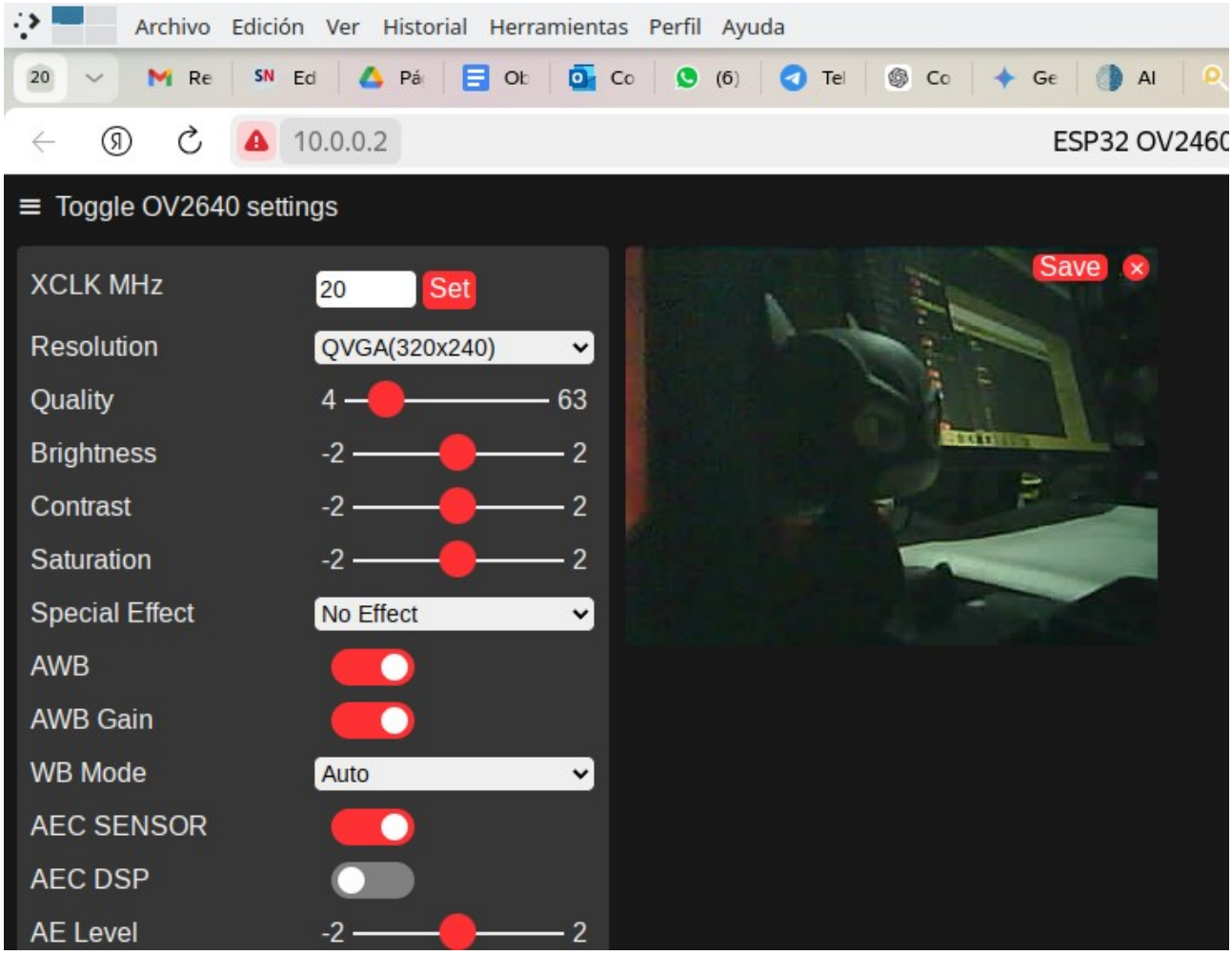


Figura 12: Acceso al servidor de video streaming del dispositivo ESP32.

Parte II: Guía Práctica para la creación de aplicaciones en Android usando OpenCV, código nativo (C++), manejo de cámara, flujo de datos y Sockets en aplicaciones web